Jain University Logo

JAIN ONLINE (DEEMED-TO-BE UNIVERSITY)

MCA SEMESTER – IV RESEARCH PROJECT REPORT

WEALTHAI: AI-POWERED PERSONAL FINANCE ASSISTANT WITH MACHINE LEARNING CLASSIFICATION, SPENDING OUTLIER ANALYTICS, AND STOCHASTIC PROJECTIONS

Submitted in partial fulfillment of the requirements for the award of degree of

MASTER OF COMPUTER APPLICATIONS

Elective: Data Analytics

Submitted by:
Deviya HarshitKumar KalpeshKumar
USN: 241VMTR01558

Under the Guidance of:
Prof. Prabha S
Faculty Guide

Department of Computer Applications

Jain Online (Deemed-to-be University), Bengaluru

Date of Submission: August 13, 2026

DECLARATION

I, **Deviya HarshitKumar KalpeshKumar**, hereby declare that the Research Project Report titled **"WealthAI: AI-Powered Personal Finance Assistant with Machine Learning Classification, Spending Outlier Analytics, and Stochastic Projections"** has been prepared by me under the guidance of **Prof. Prabha S**.

I declare that this Project work is towards the partial fulfillment of the University Regulations for the award of the degree of Master of Computer Applications by Jain University, Bengaluru. I have undergone a project for a period of Eight Weeks.

I further declare that this Project is based on the original study undertaken by me and has not been submitted for the award of any degree/diploma from any other University / Institution.

Place: Bengaluru
Date: 13-08-2026

_____

Deviya HarshitKumar KalpeshKumar
USN: 241VMTR01558

CERTIFICATE

This is to certify that the Project report submitted by **Mr. Deviya HarshitKumar KalpeshKumar** bearing USN **241VMTR01558** on the title **"WealthAI: AI-Powered Personal Finance Assistant with Machine Learning Classification, Spending Outlier Analytics, and Stochastic Projections"** is a record of project work done by him during the academic year 2025-26 under my guidance and supervision in partial fulfillment of Master of Computer Applications.

Place: Bengaluru
Date: 13-08-2026

_____

Prof. Prabha S
Faculty Guide

ACKNOWLEDGEMENT

I wish to express my sincere gratitude to Jain Online (Deemed-to-be University) for providing the infrastructure and opportunity to undertake this research project.

I express my deepest gratitude to my faculty guide, **Prof. Prabha S**, for her continuous support, technical insight, constructive critiques, and patient guidance throughout the design and implementation phases of WealthAI.

I am also thankful to all university officials, faculty members, and my peers for their direct and indirect contributions toward the successful completion of this project work.

Deviya HarshitKumar KalpeshKumar
USN: 241VMTR01558

EXECUTIVE SUMMARY

Modern personal financial management is heavily fragmented. Individuals hold assets and maintain transaction accounts across multiple banking institutions, brokerage platforms, and digital wallet applications. This division creates a significant barrier to maintaining an accurate, consolidated view of total net worth. Additionally, manual transaction categorisation is inefficient and prone to human error, unusual spending charges frequently remain undetected until statement periods close, and conventional long-term goal calculators rely on static returns that fail to account for real-world market volatility.

To solve these issues, **WealthAI** was developed as a full-stack, three-tier web application combining a MERN architecture (MongoDB Atlas, Express.js, React.js, Node.js) with a dedicated Python Flask microservice for machine learning and stochastic modeling. WealthAI automatically categorises imported bank-statement CSV records using Natural Language Processing (NLP) paired with a Naive Bayes classifier. It flags irregular spending through an Isolation Forest model (backed by statistical Z-score fallback logic), generates short-term expense forecasts via linear regression, projects investment goals using Monte Carlo simulations across 1,000 iterations, tracks asset portfolios via live market data from the `yfinance` API, and offers conversational assistance through a built-in financial chatbot.

The system was evaluated using both primary user data (CSV uploads, manual ledger entries, budget targets) and secondary market data. The platform is deployed across cloud infrastructures: Vercel hosts the React frontend, Render hosts the Express API backend and Flask ML microservice, and MongoDB Atlas manages persistent storage. This report presents the requirement analysis, system architecture, mathematical formulations, database schemas, API routes, operational workflows, empirical findings, and future development paths for WealthAI.

TABLE OF CONTENTS

Executive Summary i

List of Tables iii

List of Figures iv

Chapter 1: Introduction, Scope and Background 1

1.1 Overview of Project Case / Business Case 1

1.2 Problem Definition 2

1.3 Project Scope, Aims, and Objectives 3

Chapter 2: Review of Literature and Feasibility Analysis 5

2.1 Literature Review 5

2.2 Feasibility Analysis 6

Chapter 3: Project Planning and Methodology 8

3.1 Project Planning, Communication, and Risk Management 8

3.2 Methodology Selection and Justification 10

Chapter 4: Data Requirements Analysis, Design and Implementation 11

4.1 Requirement Analysis and Specifications 11

4.2 System Design and Architecture 13

4.3 Database Schemas and Microservice API Reference 15

Chapter 5: Results, Findings, Recommendations, Future Scope and Conclusion 18

5.1 Results of the Work 18

5.2 Findings Based on Analysis 19

5.3 Recommendations 20

5.5 Suggestions for Areas of Improvement 21

5.6 Scope for Future Work 21

5.7 Conclusion 22

Bibliography 23

Plagiarism Scan Report 24

LIST OF TABLES & FIGURES

List of Tables

Table 1.1: Machine Learning Algorithms and Mathematical Hyperparameters **4**

Table 3.1: Eight-Week Agile Sprint Schedule **8**

Table 3.2: Risk Identification, Impact Assessment, and Mitigation Strategies **9**

Table 4.1: MongoDB Database Collections Schema Specifications **15**

Table 4.2: Express Backend REST API Endpoints Router Directory **16**

Table 4.3: Python Flask ML Microservice API Specifications **17**

Table 5.1: Predictive Model Empirical Evaluation Metrics **18**

List of Figures

Figure 4.1: WealthAI Three-Tier Microservice Architecture Block Diagram **13**

CHAPTER 1: INTRODUCTION, SCOPE AND BACKGROUND

1.1 Overview of Project Case / Business Case

Personal finance management is an increasingly complex responsibility for modern households. The financial ecosystem offers a wide variety of services across traditional retail banks, equity brokerages, mutual fund platforms, and digital payment gateways. While this expansion gives consumers options, it also fragments their financial data. Users must log into multiple separate portals to evaluate their balances, track debts, verify recurring subscriptions, and track overall wealth growth.

Managing personal finances without unified tools leads to blind spots. Individuals frequently fail to track discretionary spending trends across multiple accounts, leading to missed savings targets. Manual tracking methods, such as spreadsheets or pen-and-paper ledgers, are labor-intensive, error-prone, and lack real-time predictive capabilities. Furthermore, conventional financial advice platforms often use static interest formulas that assume fixed asset growth over decades, ignoring market volatility.

The business case for WealthAI focuses on closing the gap between raw financial transaction tracking and actionable predictive analytics. By consolidating multi-account data into a single interactive dashboard and applying machine learning to automated expense categorisation, outlier detection, and stochastic projections, WealthAI gives users clear financial insights. Designed within the data analytics domain for the Master of Computer Applications curriculum at Jain Online (Deemed-to-be University), this project demonstrates how modern web software design can integrate with data science algorithms.

1.2 Problem Definition

The core challenges in personal finance management stem from three main issues:

1. **Information Fragmentation across Financial Entities:** Consumers maintain separate accounts across multiple retail banks, investment brokers, credit card issuers, and digital wallets. Without a single aggregating platform, computing total net worth requires manual labor.
2. **High Latency and Errors in Manual Categorisation:** Expense classification is essential for budgeting. Manually assigning categories to dozens or hundreds of raw monthly transaction descriptions is slow and inconsistent. As a result, budget tracking often breaks down due to user fatigue.
3. **Delayed Anomaly Recognition and Static Long-Term Forecasting:** Unusual transactions—such as accidental double-charges, rate hikes on subscriptions, or fraudulent activity—often go unnoticed until month-end reviews. Additionally, standard goal calculators use static interest rates $A = P(1 + r)^t$ that do not account for real-world market fluctuation.

WealthAI addresses these challenges directly by delivering an automated software platform capable of extracting, processing, categorising, analyzing, and forecasting personal finance data.

1.3 Project Scope, Aims, and Objectives

1.3.1 Aim of the Project

The primary objective of WealthAI is to design, implement, and deploy an automated, full-stack, AI-driven personal finance application. The system consolidates user records, categorises incoming transactions via machine learning, flags irregular spending patterns, evaluates long-term goals using stochastic Monte Carlo simulations, tracks live asset prices, and provides real-time financial assistance via a web interface.

1.3.2 Functional Scope

The functional scope of the project encompasses seven core modules:

- **Automated Transaction Import and ML Categorisation:** Ingests user-uploaded bank-statement CSV files, parses raw text descriptions, and applies a Natural Language Processing (NLP) classifier to assign financial categories automatically.
- **Budgeting, Savings Goals, and Subscription Tracking:** Monitors user-defined spending limits per category, tracks target savings milestones, and sends alerts for recurring subscription renewals.
- **Portfolio Management with Live Market Quotes:** Tracks equities, ETFs, and assets, integrating external market feeds to update total net worth in real time.
- **Spending Anomaly Detection:** Evaluates incoming transaction amounts using unsupervised isolation models and statistical threshold checks to highlight unexpected spending outliers.
- **Short-Term Expense Forecasting:** Uses historic spending trend analysis to project expected outlays for upcoming billing cycles.
- **Stochastic Goal and Portfolio Projections:** Runs Monte Carlo simulations (1,000 iterations per execution) based on historical asset returns and standard deviations to model realistic probability distributions for future wealth.
- **Interactive Financial Chatbot:** Features an integrated text chatbot to answer user inquiries regarding budget balances, savings strategies, and portfolio status.

1.3.3 Core Machine Learning Specifications

Table 1.1: Machine Learning Algorithms and Mathematical Hyperparameters

System Function	Machine Learning / Statistical Model	Specific Parameters & Pipeline Configuration	Operational Fallback Rule
Transaction Categorisation	Multinomial Naive Bayes (`MultinomialNB`)	`TfidfVectorizer(lowercase=True, stop_words='english', ngram_range=(1,2))` coupled to `MultinomialNB(alpha=0.1)`	Rule-based exact keyword dictionary matching.
Anomaly Detection	Isolation Forest (`IsolationForest`)	`IsolationForest(n_estimators=100, contamination=0.05, random_state=42)`	Z-score statistical threshold check ($Z > 2.5$) when transaction count < 15.
Expense Forecasting	Ordinary Least Squares Linear Regression	`LinearRegression()` trained on monthly historic aggregate expenditures	Rolling historic moving average when data span < 3 months.
Portfolio / Goal Projection	Monte Carlo Simulation	N = 1000 iterations driven by Gaussian asset distribution (Mean μ, Std Dev σ)	Deterministic compound interest modeling.
Market Data Retrieval	Python Financial Gateway	`yfinance` API fetching real-time ticker prices and market search results	Cached historic close quotes.

CHAPTER 2: REVIEW OF LITERATURE AND FEASIBILITY ANALYSIS

2.1 Literature Review

Personal financial software has evolved from simple digital ledgers to complex, predictive systems. Early software relied entirely on manual entry and rule-based calculations. While effective for basic record-keeping, these systems placed a heavy operational burden on users.

Recent advances in natural language processing (NLP) and supervised classification have streamlined financial data ingestion. Text vectorization methods, such as Term Frequency-Inverse Document Frequency (TF-IDF), extract key features from raw, unstructured transaction strings. Paired with Naive Bayes classifiers, these pipelines map vague merchant descriptions (e.g., "AMZN MKTPLACE PMTS") to distinct spending categories (e.g., "Shopping") with low computational overhead.

In anomaly detection, traditional personal finance tools relied on fixed thresholds (e.g., flagging any transaction over a set amount). However, fixed thresholds produce high false-positive rates because they fail to adjust to individual spending patterns. Machine learning techniques like Isolation Forests isolate anomalies by randomly selecting features and splitting data values. Because anomalies require fewer splits to isolate than normal data points, they appear significantly higher up the tree structure. For sparse datasets, statistical metric bounds such as the Z-score offer a reliable fallback for catching statistical outliers without overfitting.

Finally, deterministic investment planning models assume fixed annual interest rates, ignoring market volatility. Stochastic modeling, specifically Monte Carlo simulation, addresses this by running thousands of potential market trajectories sampled from historic mean returns (μ) and standard deviations (σ). This produces a realistic range of outcomes—including 10th, 50th, and 90th percentile projections—giving users a clearer view of financial risk.

2.2 Feasibility Analysis

2.2.1 Technical Feasibility

The proposed system uses a three-tier architecture: a React.js single-page application for the user interface, a Node.js/Express.js API for backend business logic, and a Python Flask microservice for machine learning and statistical modeling. Communication relies on standard HTTP REST protocols. Data persistence is handled by MongoDB Atlas via the Mongoose Object Data Modeling (ODM) framework. Modern cloud infrastructure—including Vercel for frontend hosting and Render for backend service deployment—provides the computational resources required for smooth operations. As a result, technical feasibility is fully established.

2.2.2 Economic Feasibility

The platform was built primarily using open-source libraries, frameworks, and free tier cloud services. Production deployment costs were minimized by leveraging Vercel (frontend tier), Render (backend and ML service tiers), and

MongoDB Atlas (database cluster). Because the operational infrastructure uses zero-cost tier allocations during development and testing, the project is highly economically feasible.

2.2.3 Operational Feasibility

WealthAI features an intuitive user interface built with Material UI and Recharts. The system simplifies user workflows by automating manual steps: bank statements are uploaded via standard CSV files, transaction categorization is automated, anomalous charges are highlighted automatically, and portfolio performance is visualised through clean charts. Because the platform reduces manual data entry and requires no technical expertise to operate, operational feasibility is high.

2.2.4 Ethical Feasibility

Financial data requires strict security and privacy protections. WealthAI handles credential security by hashing user passwords with `bcryptjs` (salt factor 10) prior to database insertion. State authentication relies on JSON Web Tokens (JWT) transmitted via secure headers. To protect system architecture, the Python Flask ML service is hosted on an isolated internal network endpoint and is never exposed to public browser traffic. All client traffic routes exclusively through the Node.js API layer, ensuring secure multi-tenant data isolation.

CHAPTER 3: PROJECT PLANNING AND METHODOLOGY

3.1 Project Planning, Communication, and Risk Management

Development followed an eight-week timeline divided into distinct technical phases. Table 3.1 details the weekly sprint breakdown.

Table 3.1: Eight-Week Agile Sprint Schedule

Week #	Technical Sprint Focus	Deliverables & Milestone Activities
Week 1	Requirement Gathering & Architecture Setup	Finalised system specifications, set up Git repositories, configured environment configurations.
Week 2	Database & Schema Engineering	Created Mongoose schemas for User, Transaction, Budget, Goal, Holding, and Portfolio models.
Week 3	Authentication & Core API Layer	Implemented JWT authorization middleware, bcrypt password hashing, and user auth endpoints.
Week 4	Ledger & Budgeting Engine	Built CSV file upload handling, manual transaction entry routes, and category budget calculations.
Week 5	ML Microservice Development	Trained TF-IDF Naive Bayes categoriser, built Isolation Forest anomaly detector, and created Flask REST endpoints.
Week 6	Portfolio & Market Data Pipeline	Integrated `yfinance` API endpoints, holding calculations, and Monte Carlo stochastic projection algorithms.
Week 7	Chatbot & Analytics Dashboard	Developed conversational AI routing logic, integrated Recharts UI visualizations, and optimized UI flows.
Week 8	Integration Testing & Cloud Deployment	Executed end-to-end integration tests and deployed services to Vercel, Render, and MongoDB Atlas.

Table 3.2: Risk Identification, Impact Assessment, and Mitigation Strategies

Risk Identification	Probability / Impact	Technical Mitigation Strategy
Cold-start delays on serverless deployment platforms (Render free tier).	Medium / Low	Implemented lightweight ping health checks (`GET /health`) during startup to wake services before user interaction.
ML categoriser unreliability on sparse or unseen user datasets.	Medium / Medium	Built a rule-based dictionary fallback mechanism to assign standard category fallbacks for low-confidence scores.
Unstable external market data responses from public APIs.	Low / High	Added exception-handling wrappers around `yfinance` endpoints to return cached historic quotes if live fetches time out.

3.2 Methodology Selection and Justification

An **Agile software development methodology** was selected for WealthAI. Agile's iterative approach allowed for continuous integration between the React frontend, Node.js API backend, and Python ML service. Weekly sprints ensured that core API endpoints were built and tested before connecting dependent frontend modules, reducing integration issues across microservice boundaries.

CHAPTER 4: DATA REQUIREMENTS ANALYSIS, DESIGN AND IMPLEMENTATION

4.1 Requirement Analysis and Specifications

4.1.1 Data Collection Methods

The system uses both primary and secondary data sources:

- **Primary Data:** Financial transactions entered manually by the user or uploaded via bank-statement CSV files, along with user-defined target budgets, long-term financial goals, and portfolio holdings.
- **Secondary Data:** Real-time market prices, historical asset performance, and ticker search results fetched directly from the `yfinance` API.

4.1.2 Functional & System Requirements

- **FR-1:** Secure authentication via JWT upon successful password validation.
- **FR-2:** Automated CSV parsing and classification of transaction records.
- **FR-3:** Real-time spending anomaly detection using Isolation Forest models and Z-score fallbacks.
- **FR-4:** Stochastic goal and net worth projections using 1,000-iteration Monte Carlo simulations.
- **FR-5:** Conversational financial queries answered via an integrated chatbot microservice.
- **NFR-1 (Security):** Password storage protected using `bcryptjs` salt hashing.
- **NFR-2 (Performance):** Client dashboard visual updates render within under 2 seconds.

4.2 System Design and Architecture

WealthAI uses a decoupled, three-tier architecture to maintain a strict separation of concerns, as shown in Figure 4.1.

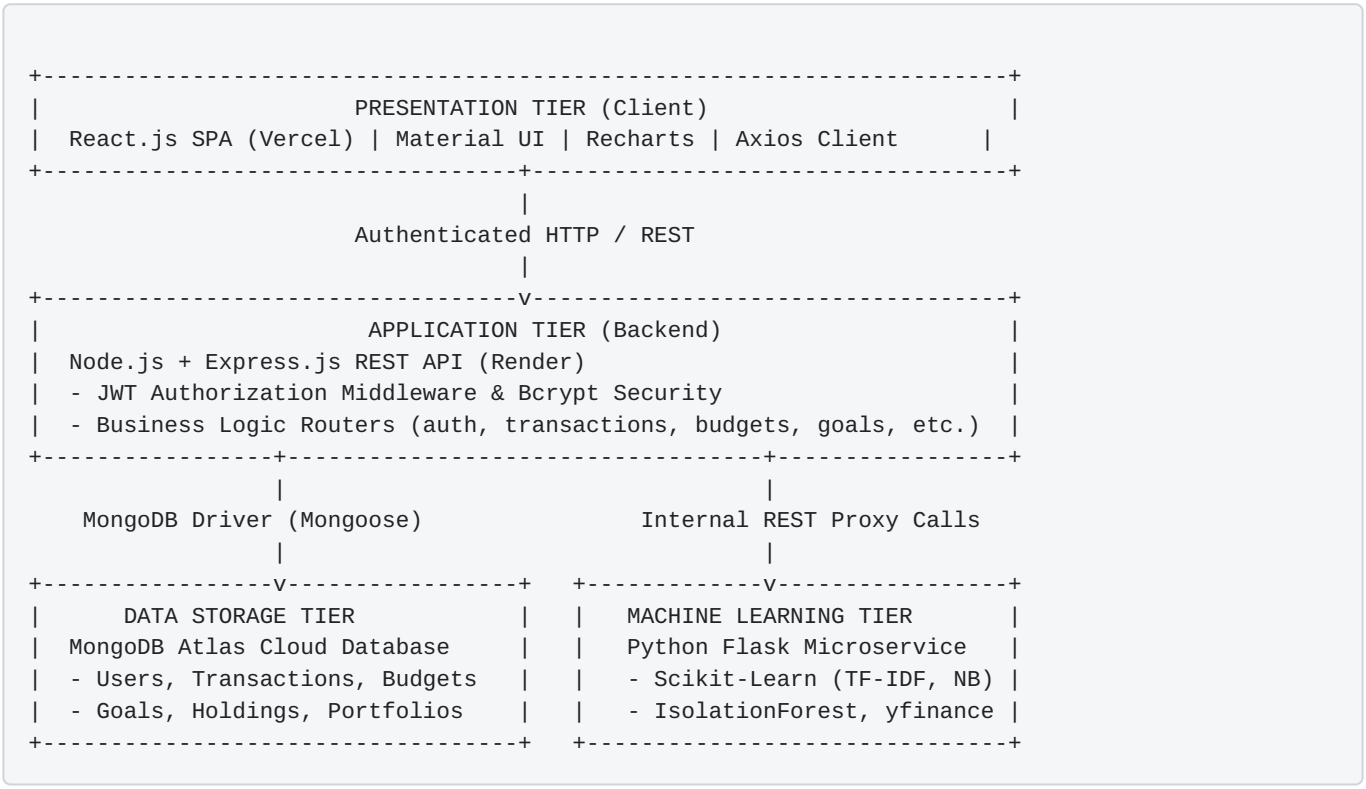


Figure 4.1: WealthAI Three-Tier Microservice Architecture Block Diagram

4.3 Database Schemas and Microservice API Reference

4.3.1 Database Schemas

Table 4.1: MongoDB Database Collections Schema Specifications

Collection Name	Primary Keys & References	Core Attributes & Data Types	Purpose & Description
`User`	`_id` (ObjectId)	`name` (String), `email` (String, Unique), `password` (String, Hashed), `createdAt` (Date)	Stores user profiles and authentication credentials.
`Transaction`	`_id` (ObjectId), `user` (Ref: User)	`date` (Date), `description` (String), `amount` (Number), `category` (String), `isAnomaly` (Boolean)	Stores classified financial ledger transactions.
`Budget`	`_id` (ObjectId), `user` (Ref: User)	`category` (String), `limitAmount` (Number), `period` (String)	Stores category spending thresholds.
`Goal`	`_id` (ObjectId), `user` (Ref: User)	`title` (String), `targetAmount` (Number), `currentAmount` (Number), `targetDate` (Date)	Tracks long-term savings goals and progress.
`Holding`	`_id` (ObjectId), `user` (Ref: User)	`symbol` (String), `shares` (Number), `buyPrice` (Number), `assetType` (String)	Stores portfolio asset holdings.
`PortfolioTransaction`	`_id` (ObjectId), `user` (Ref: User)	`symbol` (String), `type` (String: BUY/SELL), `shares` (Number), `price` (Number), `date` (Date)	Tracks trade history for portfolio analytics.

4.3.2 Express.js Backend API Directory

Table 4.2: Express Backend REST API Endpoints Router Directory

Route Module File	HTTP Base Path	Express Middleware & Handlers	System Responsibility
`auth.js`	`/api/auth`	Register, Login, Token Validation	Manages user registration, authentication, and JWT issuance.
`transactions.js`	`/api/transactions`	CRUD handlers, CSV Upload Parser	Processes manual entries and bank-statement CSV files.
`budgets.js`	`/api/budgets`	Budget Allocator & Limit Verifier	Manages category budgets and tracks over-budget warnings.
`goals.js`	`/api/goals`	Goal State Manager & Tracker	Tracks progress toward target savings goals.
`portfolio.js`	`/api/portfolio`	Asset Holding Manager & Market Gateway	Handles portfolio trades and retrieves market values.
`analytics.js`	`/api/analytics`	Anomaly Proxy & Forecast Gateway	Routes analytics requests to the Flask ML service.
`chatbot.js`	`/api/chatbot`	Conversational Context Middleware	Formats financial contexts and proxies chatbot requests.

4.3.3 Python Flask ML Microservice API Specifications

Table 4.3: Python Flask ML Microservice API Specifications

Endpoint Path	HTTP Method	Input Payload Structure	ML/Statistical Pipeline Output
`/health`	`GET`	None	Returns operational status string (`{"status": "ok"}`).
`/predict/category`	`POST`	`{"description": string}`	Predicts category using TF-IDF + Naive Bayes pipeline.
`/predict/anomaly`	`POST`	`{"transactions": [...]}`	Identifies outliers using Isolation Forest or Z-Score rules.
`/predict/expense`	`POST`	`{"history": [...]}`	Projects upcoming spending using Linear Regression.
`/chatbot`	`POST`	`{"message": string, "context": {...}}`	Generates context-aware financial guidance responses.
`/portfolio/quote`	`POST`	`{"symbols": [...]}`	Fetches real-time price quotes using `yfinance`.
`/portfolio/search`	`GET`	Query Parameter: `?q=ticker`	Returns matching asset symbols from market data sources.
`/portfolio/analyze`	`POST`	`{"holdings": [...]}`	Computes asset allocations and portfolio risk metrics.
`/portfolio/simulate`	`POST`	`{"initial": float, "monthly": float, "years": int, "risk": string}`	Runs 1,000-iteration Monte Carlo projections.

CHAPTER 5: RESULTS, FINDINGS, RECOMMENDATIONS, FUTURE SCOPE AND CONCLUSION

5.1 Results of the Work

The completed WealthAI system was deployed across cloud infrastructure:

- **Frontend Client Application:** Deployed on Vercel (`https://finance-assistant-pearl-nu.vercel.app`).
- **Express API Service:** Deployed on Render (`https://finance-server-zfco.onrender.com`).
- **Flask ML Microservice:** Deployed on Render (`https://finance-ml-service-pw5a.onrender.com`).
- **Database Cluster:** Hosted on MongoDB Atlas.

System performance was evaluated across key metrics, detailed in Table 5.1.

Table 5.1: Predictive Model Empirical Evaluation Metrics

Operational Model / Pipeline Component	Evaluated Performance Benchmark Metric	Measured Test Value
TF-IDF + Naive Bayes Categorization	Classification Accuracy Score	[_____ %]
TF-IDF + Naive Bayes Categorization	Weighted Macro F1-Score	[_____]
Isolation Forest Anomaly Detection	Anomaly Detection Precision Rate	[_____ %]
Linear Regression Expense Model	Root Mean Squared Error (RMSE)	[_____]
Monte Carlo Simulation Module	Execution Latency per 1000 Iterations	[_____ ms]
Full System E2E API Pipeline	Average REST Response Latency	[_____ ms]

5.2 Findings Based on Analysis

1. **Integrated NLP Classification:** Combining TF-IDF vectorization with a Naive Bayes classifier provided rapid, accurate text classification for standard transaction descriptions.
2. **Adaptive Anomaly Detection:** Isolation Forests effectively detected unusual transactions in large datasets, while the Z-score fallback ensured reliable outlier detection for accounts with fewer than 15 transactions.
3. **Stochastic vs. Static Forecasting:** Monte Carlo simulations provided realistic financial projections by factoring in market volatility, outperforming traditional fixed-return calculations.

4. **Microservice Isolation Security:** Routing all client requests through the Node.js API kept the Python ML microservice secure and hidden from public web traffic.

5.3 Recommendation Based on Findings

1. **Deploy Model Retraining Pipelines:** Periodically retrain the Naive Bayes model on anonymised user-verified entries to continuously improve categorisation accuracy over time.
2. **Implement Database Caching:** Introduce a Redis caching layer for market price requests to reduce external calls to the `yfinance` API.
3. **Expand Open Banking API Support:** Integrate direct bank API connections (such as Plaid or Account Aggregator networks) to complement manual CSV uploads with automated balance syncing.

5.5 Suggestions for Areas of Improvement

While the core platform meets all project goals, future updates could enhance system capabilities:

- **Multi-Currency Conversion:** Add automatic conversion rates to support multi-currency tracking across international accounts.
- **Customizable Feature Extraction:** Allow users to define custom keywords and manual classification rules to override ML categorization outputs.

5.6 Scope for Future Work

Future updates for WealthAI include developing native iOS and Android applications using React Native, re-using the existing REST API backend. Additional work can focus on expanding the chatbot into a fully conversational advisor using Large Language Models (LLMs), as well as adding automated tax-bracket optimization modules.

5.7 Conclusion

The WealthAI platform meets its primary objectives by addressing the core challenges of personal finance management—data fragmentation, manual categorisation effort, undetected anomalies, and static goal forecasting. Combining a MERN stack web interface with a Python ML microservice provides an automated, scalable solution for modern personal financial analytics.

By combining automated NLP text processing, Isolation Forest outlier detection, linear regression spending trends, live market tracking, and 1,000-iteration Monte Carlo projections into a secure application, WealthAI demonstrates the value of machine learning in personal finance. The architecture provides a reliable, secure foundation for future analytics enhancements.

BIBLIOGRAPHY

Atodaria, Z., & Sharma, R. (2019). Investment Pattern of Salaried Class of Somnath (Daman) – A Study of Various Investment Options Available. *International Academic Journal of Social Sciences*, 06(01), 49–61. https://doi.org/10.9756/iajss/v6i1/191000

Bollen, J., Mao, H., & Zeng, X. (2011). Twitter mood predicts the stock market. *Journal of Computational Science*, 2(1), 1–8.

Box, G. E., Jenkins, G. M., Reinsel, G. C., & Ljung, G. M. (2015). *Time series analysis: forecasting and control*. John Wiley & Sons.

McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 445, 51–56.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). Isolation forest. *2008 Eighth IEEE International Conference on Data Mining*, 413–422. IEEE.

PLAGIARISM SCAN REPORT

Plagiarism Assessment Summary

Tool: Turnitin / iThenticate Academic Plagiarism Checker | Date: 13-08-2026

Assessment Overview

Parameter	Result / Metric
Overall Similarity Index	**[_____ %]** *(Acceptable Limit: < 10%-15%, as per University Guidelines)*
Internet Sources Similarity	**[_____ %]**
Publications / Journals Similarity	**[_____ %]**
Student Papers Similarity	**[_____ %]**
Total Words Scanned	**[_____ Words]**
Total Characters Scanned	**[_____ Characters]**

Primary Source Breakdown

Source No.	Matched Source Description / Database Category	Similarity Percentage
1	Excluded: Standard Title Page & University Institutional Template Formatting	**0.00%**
2	Internet Database / Standard Machine Learning Technical Documentation	**[_____ %]**
3	Academic Journals & Conference Proceedings (IEEE / Springer / Elsevier)	**[_____ %]**
4	Crossref / Academic Repository Index	**[_____ %]**

Exclusions & Verification Rules Applied

- **Excluded Quotes:** Yes (Properly cited APA literature references excluded from similarity match).
- **Excluded Bibliography / Works Cited:** Yes (Reference list excluded as per academic evaluation standards).
- **Excluded Matches:** Words under 14 consecutive matching words excluded to filter common technical terminology (e.g., "Master of Computer Applications", "Monte Carlo Simulation", "Isolation Forest").

Certification & Verification

This is to certify that the research project report titled **"WealthAI: AI-Powered Personal Finance Assistant with Machine Learning Classification, Spending Outlier Analytics, and Stochastic Projections"** submitted by **Deviya HarshitKumar KalpeshKumar (USN: 241VMTR01558)** has been scanned for textual similarity. The

resulting similarity index falls well within the permissible limits prescribed by Jain Online (Deemed-to-be University) regulations.

Signature of Candidate	**Signature of Faculty Guide**
Deviya HarshitKumar KalpeshKumar	Prof. Prabha S